



Numerical Representation of Numbers

Binary · Octal · Decimal · Hexadecimal · Complements · BCD & More

90-Minute Lecture | Computer Organization & Architecture

Mansur Mulk

North South University



Mansur Mulk

North South University

Course Agenda

01

Number Systems Overview

Why different bases matter

02

Binary Numbers

Base-2 representation

03

Octal Numbers

Base-8 representation

04

Decimal Numbers

Our everyday base-10

05

Hexadecimal Numbers

Base-16 in computing

06

Conversions

Between all bases

07

Complements (1s & 2s)

Signed arithmetic

08

Binary Addition & Subtraction

Arithmetic operations

09

BCD & BCD Addition

Binary Coded Decimal

10

Other Systems

Gray code, ASCII, etc.



What is a Number System?

A number system represents quantities using a set of symbols and positional rules.

Base / Radix

The number of unique digits. Base-10 uses 0–9.

Positional Value

Each digit's value depends on its column position.

Symbol Set

Different bases use different digit symbols (e.g., A–F in hex).

General Positional Notation:

$$N = d_n \times B^n + d_{n-1} \times B^{n-1} + \dots + d_1 \times B^1 + d_0 \times B^0$$

Mansur Mulk

North South University



Section

Binary Numbers

Base-2 · The language of computers



Binary Numbers — The Basics

Binary (Base-2) uses only two digits: **0** and **1**

Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Weight (Dec)	128	64	32	16	8	4	2	1
Example Bit	1	0	1	1	0	1	0	1

Example: Convert 10110101_2 to Decimal

$$= 128 + 0 + 32 + 16 + 0 + 4 + 0 + 1 = 181_{10}$$

LSB = rightmost bit | MSB = leftmost bit | n-bit range: 0 to $2^n - 1$



Binary → Decimal Conversion

Method: Expand by positional powers of 2, then sum.

1011_2

$$= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$= 11_{10}$$

11001_2

$$= 1 \times 2^4 + 1 \times 2^3 + 0 + 0 + 1 \times 2^0$$

$$= 25_{10}$$

1111111_2

$$= 64 + 32 + 16 + 8 + 4 + 2 + 1$$

$$= 127_{10}$$



Decimal → Binary Conversion

Method: Repeated Division by 2 — record remainders bottom to top.

Example: $45_{10} = ?_2$

Division	Quotient	Remainder
$45 \div 2$	22	1 ← LSB
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1 ← MSB

Read remainders bottom to top:

$$45_{10} = 101101_2$$

Verify: $32+0+8+4+0+1 = 45 \checkmark$

Mansur Mulk
North South University



Section

Octal Numbers

Base-8 · Compact binary shorthand



Octal Numbers (Base-8)

Octal uses 8 digits: **0, 1, 2, 3, 4, 5, 6, 7** Each octal digit = exactly 3 binary bits

Octal	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111

Octal → Decimal | Example: 352_8

$$3 \times 64 + 5 \times 8 + 2 \times 1 = 234_{10}$$

Octal ↔ Binary | Example: 273_8

$$2 \rightarrow 010 \quad 7 \rightarrow 111 \quad 3 \rightarrow 011 = 010111011_2$$

Decimal → Octal: Divide by 8, read remainders bottom-up.

$$255_{10}: 255 \div 8 = 31 \text{ R}7 \rightarrow 31 \div 8 = 3 \text{ R}7 \rightarrow 3 \div 8 = 0 \text{ R}3 \rightarrow 377_8$$

Mansur Mulk

North South University



Section

Hexadecimal Numbers

Base-16 · Programmer's favorite



Hexadecimal Numbers (Base-16)

Hex uses 16 symbols: **0–9 and A–F** (1 hex digit = 4 binary bits = 1 nibble)

Dec	Hex	Bin	Dec	Hex	Bin
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	10	A	1010
3	3	0011	11	B	1011
4	4	0100	12	C	1100
5	5	0101	13	D	1101
6	6	0110	14	E	1110
7	7	0111	15	F	1111

Hex → Decimal

1A3F₁₆ = ?

$$\begin{aligned} &1 \times 4096 + 10 \times 256 + 3 \times 16 + 15 \\ &= 4096 + 2560 + 48 + 15 \\ &= 6719_{10} \end{aligned}$$

Code prefix: 0x1A3F or #1A3F



Hex $\leftrightarrow$ Binary Conversion

1 hex digit = exactly 4 binary bits (nibble). Group bits in fours from right.

Binary $\rightarrow$ Hex | Group from right into nibbles:

$$\begin{array}{ccccccc} & & 1101 & 0101 & 1010 & 1110_2 & \\ \rightarrow & D & 5 & A & E & = & D5AE_{16} \end{array}$$

Hex $\rightarrow$ Binary | Expand each digit to 4 bits:

$$\begin{array}{l} 3F8C_{16} \rightarrow 3=0011 \quad F=1111 \quad 8=1000 \quad C=1100 \\ \quad \quad \quad = 0011111110001100_2 \end{array}$$

Mansur Mulk
North South University



Section

Base Conversions

Moving between all number systems



Conversion Methods — Roadmap

All conversions route through Decimal or Binary as an intermediate step.

Binary
(Base 2)

Octal
(Base 8)

Decimal
(Base 10)

Hexadecimal
(Base 16)

Any → Decimal

Expand using positional weights, then sum

Decimal → Binary

Divide by 2, collect remainders (LSB first)

Decimal → Octal

Divide by 8, collect remainders (LSB first)

Decimal → Hex

Divide by 16, collect remainders (LSB first)

Binary ↔ Octal

Group binary bits in 3s

Binary ↔ Hex

Group binary bits in 4s



Worked Conversion Examples

Binary
→ Octal

101110011_2

Group in 3s: 101 110 011

$= 563_8$

Hex
→ Binary

$A9_{16}$

A=1010, 9=1001

$= 10101001_2$

Decimal
→ Hex

255_{10}

$255 \div 16 = 15 \text{ R } 15(F), 15 \div 16 = 0 \text{ R } 15(F)$

$= FF_{16}$

Octal
→ Decimal

742_8

$7 \times 64 + 4 \times 8 + 2 \times 1 = 448 + 32 + 2$

$= 482_{10}$

Mansur Mulk
North South University



Section

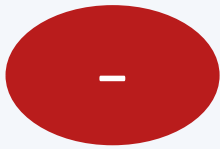
Complements

1's Complement & 2's Complement



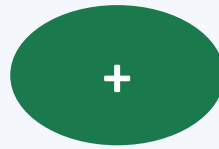
Why Do We Need Complements?

Complements allow computers to perform subtraction using addition hardware.



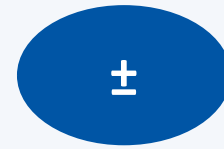
Subtraction Problem

Hardware subtraction circuits are complex. Complements simplify this.



Addition Solution

Using complements, subtraction becomes addition of a complement.



Signed Numbers

Complements give a clean way to represent negative numbers in binary.

Two types: $(r-1)$'s complement and r 's complement (most common: 1's and 2's for binary)



1's Complement

Rule: Invert every bit ($0 \rightarrow 1$, $1 \rightarrow 0$).

Example 1:



Example 2:



Example 3:



⚠ Two zeros: $+0 = 00000000$, $-0 = 11111111$ (limitation of 1's complement)



2's Complement

Rule: Invert all bits (1's complement), then add 1.

Example: Find 2's complement of 01001101_2

Step 1

Original number

0 1 0 0 1 1 0 1

Step 2

1's Complement (invert)

1 0 1 1 0 0 1 0

Step 3

Add 1 → Result

1 0 1 1 0 0 1 1

✓ Only ONE zero. Standard for all modern CPUs!



2's Complement — Signed Integers

For n-bit 2's complement: Range = $-2^{(n-1)}$ to $+2^{(n-1)} - 1$

Bit Pattern	Unsigned	Signed (2's comp)
0000	0	+0
0001	1	+1
0010	2	+2
0011	3	+3
0100-0111	4-7	+4 to +7
1000	8	-8 (most negative)
1001	9	-7
1010	10	-6
1111	15	-1

MSB=0 → positive

MSB=1 → negative

Negate: invert+1

One zero only

8-bit: -128 to +127

16-bit: -32768 to +32767

Mansur Mulk

North South University



Section

Binary Arithmetic

Addition & Subtraction



Binary Addition — Rules & Examples

Binary addition follows 4 simple rules with carry propagation.

A	B	Cin	Sum	Cout
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1 ← Carry
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1 ← Carry

Example:

$$\begin{array}{r} 110101_2 = 53 \\ + 011011_2 = 27 \\ \hline 100000_2 = 80 \end{array}$$

Overflow: Carry into MSB $\neq$ Carry out of MSB $\rightarrow$ signed overflow



Binary Subtraction via 2's Complement

$A - B = A + (2\text{'s complement of } B)$. No separate subtraction circuit needed!

Example: $53 - 27 = ?$ (8-bit 2's complement)

1 $53_{10} = 00110101_2$

0 0 1 1 0 1 0 1

2 $27_{10} = 00011011_2$

0 0 0 1 1 0 1 1

3 1's comp of 27

1 1 1 0 0 1 0 0

4 2's comp of 27 (+1)

1 1 1 0 0 1 0 1

5 Add the two values

1 0 0 0 1 1 0 1 0

6 Discard carry $\rightarrow 26_{10} \checkmark$

0 0 0 1 1 0 1 0



Overflow in Binary Arithmetic

Overflow occurs when the result exceeds the representable range.

Overflow $\Leftrightarrow$ Carry into MSB $\neq$ Carry out of MSB

No Overflow (OK)

01000000 (+64)
+ 00111111 (+63)
= 01111111 (+127)

Cin=0, Cout=0 $\rightarrow$ Equal $\checkmark$

OVERFLOW! (+127+1)

01111111 (+127)
+ 00000001 (+1)
= 10000000 (-128!)

Cin=1, Cout=0 $\rightarrow$ Diff $\times$

Mansur Mulk

North South University



Section

BCD

Binary Coded Decimal



Binary Coded Decimal (BCD)

BCD encodes each decimal digit independently as a 4-bit binary group (0000–1001).

Decimal	BCD	Decimal	BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

⚠ BCD codes 1010–1111 are INVALID

Convert 396_{10} to BCD:

3 9 6
0011 1001 0110

BCD → Decimal: 0111 0100 0010

0111=7, 0100=4, 0010=2 → 742_{10}



BCD Addition

BCD addition requires a correction step when the sum > 9 or a carry occurs.

Algorithm:

1. Add the two BCD digits as normal binary.
2. If result > 9 (1001) OR carry generated: Add 6 (0110) to skip invalid states.
3. Any carry propagates to the next BCD group.

Case 1: No Correction (5+3=8)

```
  0101  (5)
+ 0011  (3)
-----
  1000  (8) ← Valid ✓
```

Case 2: Correction Needed (7+8=15)

```
  0111+1000 = 1111 (>9!)
+ 0110  (correction)
-----
 1 0101 → carry=1, digit=5 → 15 ✓
```



BCD — Advantages & Disadvantages

BCD is widely used where exact decimal representation is critical.

✓ Advantages

✓ No rounding errors for decimal arithmetic

✓ Simple decimal conversion

✓ 7-segment display compatible

✓ Financial & banking systems

✓ Accurate currency calculations

✗ Disadvantages

✗ 6 of 16 combinations wasted

✗ Less efficient than pure binary

✗ Requires correction step

✗ Larger storage requirement

✗ Slower arithmetic operations

Mansur Mulk

North South University



Section

Other Number Systems

Gray Code · ASCII · Excess-3 · More



Gray Code

Gray code: consecutive values differ in exactly ONE bit — minimizes transition errors.

Decimal	Binary	Gray Code
0	0000	0000
1	0001	1000
2	0010	1100
3	0011	0100
4	0100	0110
5	0101	1110
6	0110	1010
7	0111	0010

Binary → Gray:

$G[\text{MSB}] = B[\text{MSB}]$
 $G[i] = B[i] \text{ XOR } B[i-1]$
(XOR adjacent binary bits)

Example: Binary 1101 → Gray Code

B: 1 1 0 1
G: 1 0 1 1



Excess-3 (XS-3) Code

Each decimal digit d is encoded as $(d + 3)$ in binary. Self-complementing code.

Decimal	BCD	Excess-3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Self-complementing

9's complement found by inverting bits

No zero code

Code 0011 represents digit 0

Symmetric pairs

(0,9),(1,8),(2,7)... are bit complements

Carry propagation

Simpler addition in some cases



ASCII — American Standard Code for Information Interchange

ASCII maps characters to 7-bit codes (0–127). Extended ASCII uses 8 bits (0–255).

Char	Dec	Hex	Char	Dec	Hex
'A'	65	41	'a'	97	61
'B'	66	42	'b'	98	62
'Z'	90	5A	'z'	122	7A
'0'	48	30	'9'	57	39
Space	32	20	'!'	33	21
NUL	0	00	DEL	127	7F

0–31	Control characters (NUL, CR, LF...)
32	Space
48–57	Digits '0'–'9'
65–90	Uppercase A–Z
97–122	Lowercase a–z
127	DEL (127)

Unicode extends to 1,114,112 code points. UTF-8 is backward-compatible with ASCII for first 128 chars.



Signed Magnitude Representation

Simplest signed representation: MSB = sign bit (0=+, 1=-), remaining bits = magnitude.

MSB = 0 → Positive | MSB = 1 → Negative | Magnitude in remaining bits

0 0 1 1 0 1 0 1

+53₁₀

1 0 1 1 0 1 0 1

-53₁₀

0 0 0 0 0 0 0 0

+0₁₀

1 0 0 0 0 0 0 0

-0₁₀ ← Problem!



Number System Comparison

Summary of all positional number systems in computing.

Property	Binary	Octal	Decimal	Hexadecimal
Base (Radix)	2	8	10	16
Digits Used	0,1	0–7	0–9	0–9, A–F
Bits per digit	1	3	~3.32	4
Common Usage	CPU/Memory	Unix perms	Everyday	Memory addrs
Prefix notation	0b	0	none	0x
4-bit max	1111	17	15	F
8-bit max	11111111	377	255	FF



Binary Fractions

Binary fractions use negative powers of 2 to the right of the binary point.

$$101.011_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} = 5.375_{10}$$

Decimal → Binary Fraction: Repeated $\times 2$, collect integer parts.

$$0.6875 \times 2 = 1.375 \rightarrow 1$$

$$0.375 \times 2 = 0.75 \rightarrow 0$$

$$0.75 \times 2 = 1.5 \rightarrow 1$$

$$0.5 \times 2 = 1.0 \rightarrow 1 \text{ (done)}$$

$$0.6875_{10} = 0.1011_2$$



Hexadecimal in Memory Addressing

Hex is the standard notation for memory addresses and machine code.

0x00000000	Start of memory space
0x0000FFFF	First 64 KB
0x7FFFFFFF	2 GB address (32-bit)
0xFFFFFFFF	4 GB max (32-bit address space)
0x00007FFF FFFFFFFF	User space limit (64-bit Linux)



Practice Problems — Conversions

Work these out — answers on the next slide!

- 1 Convert 11010110_2 to Decimal
- 2 Convert 189_{10} to Binary
- 3 Convert $5E7_{16}$ to Decimal
- 4 Convert 372_8 to Hexadecimal
- 5 Convert 0.8125_{10} to Binary fraction
- 6 Find 2's complement of 10110100_2
- 7 BCD Add: $0100\ 0110 + 0011\ 0111$
- 8 Convert 10110111_2 to Gray code



Answers — Practice Problems

Verify your work:

P1	$11010110_2 = 128+64+16+4+2 = 214_{10}$
P2	$189_{10} = 10111101_2$
P3	$5E7_{16} = 5 \times 256 + 14 \times 16 + 7 = 1511_{10}$
P4	$372_8 = 011\ 111\ 010_2 = FA_{16}$
P5	$0.8125_{10} = 0.1101_2$
P6	2's comp of $10110100 = 01001100$
P7	BCD sum = $1000\ 0011$ (83)
P8	$10110111_2 \rightarrow \text{Gray} = 11101100$



Hamming Code — Error Detection & Correction

Hamming code adds parity bits to detect and correct single-bit errors in transmitted data.

Parity bit positions	Placed at positions 1, 2, 4, 8, ... (powers of 2)
Coverage rule	Each parity bit covers specific data bit positions via its binary value
Error correction	Syndrome (XOR of failed positions) gives exact error bit location
Extended Hamming	Add overall parity bit for double-bit detection (SECDED)



IEEE 754 Floating Point Representation

IEEE 754 is the international standard for floating-point arithmetic in computers.



$$\text{Value} = (-1)^S \times 2^{(E-127)} \times 1.\text{Mantissa}$$

Single precision (32-bit)	Range $\sim \pm 3.4 \times 10^{38}$, Precision ~ 7 decimal digits
Double precision (64-bit)	Sign:1, Exp:11, Mantissa:52 bits, ~ 15 significant digits
Special values	$\pm \infty$ (exp=255, mant=0), NaN (exp=255, mant $\neq$ 0)
Normalized	Implied leading 1 in mantissa ($1.\text{fraction} \times 2^{\text{exp}}$)



Generalized: r 's and $(r-1)$'s Complement

Complement theory applies to any base r , not just binary.

$(r-1)$'s Complement

Subtract each digit from $(r-1)$.
Base-10 $\rightarrow$ 9's complement
Base-2 $\rightarrow$ 1's complement

Example (9's comp of 3704):
 $9999 - 3704 = 6295$

r 's Complement

$(r-1)$'s complement + 1.
Base-10 $\rightarrow$ 10's complement
Base-2 $\rightarrow$ 2's complement

Example (10's comp of 3704):
 $10000 - 3704 = 6296$

Subtraction using complements: $A - B = A + \text{complement}(B)$, discard final carry.



Bitwise Operations

Bitwise operations act on individual bits — fundamental in computing.

AND
&

1 only if BOTH bits are 1

1011 AND 0110 = 0010

OR
|

1 if at least ONE bit is 1

1011 OR 0110 = 1111

XOR
^

1 if bits are DIFFERENT

1011 XOR 0110 = 1101

NOT
~

INVERT every bit

NOT 10110101 = 01001010



Shift Operations in Binary

Shifts multiply or divide by powers of 2 — fast hardware operations.

Logical Left Shift (<<)

×2; zeros fill right; MSB lost

0 0 1 1 0 1 1 0 → 0 1 1 0 1 1 0 0

Logical Right Shift (>>)

÷2; zeros fill left; LSB lost

0 0 1 1 0 1 1 0 → 0 0 0 1 1 0 1 1

Arithmetic Right Shift

÷2 signed; sign bit copies

1 0 1 1 0 1 1 0 → 1 1 0 1 1 0 1 1

Circular Rotate (ROL/ROR)

Bits wrap; no loss

0 0 1 1 0 1 1 0 → 0 1 1 0 1 1 0 0 (wrap)



Number Systems in Real Computers

How all these concepts come together in real computing hardware.

Binary	CPU Registers, ALU, RAM, all hardware logic
Hexadecimal	Machine code, addresses, color codes (#FF5733), MACs
Octal	Unix/Linux file permissions (chmod 755 = rwxr-xr-x)
BCD	Digital clocks, calculators, banking software, ATMs
2's Comp	All signed integer arithmetic in modern CPUs (x86, ARM)
IEEE 754	All floating-point: Python float, C double, GPU shaders
Gray Code	Rotary encoders, error-tolerant sensors, Karnaugh maps
ASCII/Unicode	Text in files, keyboard input, network protocols



Quick Reference — Key Formulas

Essential formulas for the exam:

Range (unsigned n-bit)	$0 \text{ to } 2^n - 1$
Range (signed 2's comp n-bit)	$-2^{(n-1)} \text{ to } +2^{(n-1)} - 1$
2's Complement of X	Invert all bits, then add 1
Binary $\rightarrow$ Gray	$G[n] = B[n] \text{ XOR } B[n-1]$
BCD correction	If sum > 9: add 0110
Hex $\leftrightarrow$ Binary	Group 4 bits per hex digit
Octal $\leftrightarrow$ Binary	Group 3 bits per octal digit
IEEE 754 (32-bit)	$(-1)^S \times 2^{(E-127)} \times 1.M$

Mansur Mulk

North South University



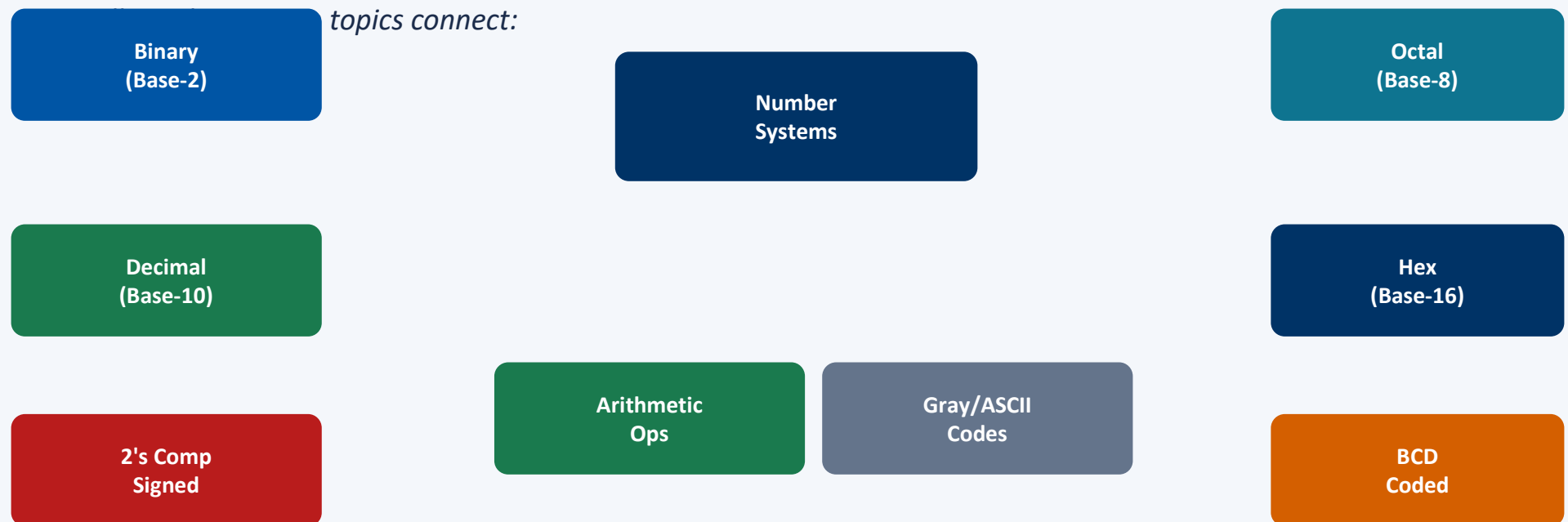
Section

Review & Summary

Putting it all together



Concept Map — All Topics





Exam Tips & Common Mistakes

What to watch for in tests and real applications:

✓ DO	Always write the base subscript (e.g., 1011_2 , 255_{10} , FF_{16})
✓ DO	Check BCD sums — add 6 correction if > 9 or carry occurs
✓ DO	Verify conversions by converting back to the original
✗ DON'T	Confuse 1's complement (just invert) with 2's complement (invert+1)
✗ DON'T	Forget to discard the final carry in 2's complement subtraction
✗ DON'T	Mix up LSB and MSB positions when grouping bits for conversion
⚡ TIP	For hex $\leftrightarrow$ binary, memorize the 16 nibble patterns (0000 to 1111)
⚡ TIP	2's complement range: $-2^{(n-1)}$ to $+2^{(n-1)}-1$ (one extra negative!)



Further Reading & Resources

Deepen your understanding with these resources:

Digital Design

M. Morris Mano & Michael D. Ciletti

Standard text, Ch. 1–2 on number systems

Computer Organization & Architecture

William Stallings

Deep coverage of data representation

Logic and Computer Design Fundamentals

Mano & Kime

Excellent worked examples throughout

The Art of Electronics

Horowitz & Hill

Practical binary & hex in circuits



Thank You!

Mansur Mulk

North South University

Key Takeaways

- Binary is the foundation — all other bases convert through it
- 2's complement is the universal signed integer representation
 - Hex is compact shorthand for binary (4 bits per digit)
- BCD preserves decimal precision at the cost of efficiency
- Complements elegantly turn subtraction into addition